AC2 - Questões teóricas (Especial)

1) O espaço de endereçamento é um conjunto de endereços disponíveis que um programa ou processo pode referenciar.

2) Descodificação de endereços.

3)

a) Cada byte (8 bits) tem o seu único endereço que pode ser acedido individualmente. A unidade de memória mais pequena que pode ser endereçado é o byte.

b) Cada bit tem o seu único endereço que pode ser acedido individualmente. A unidade de memória mais pequena que pode ser endereçado é o bit.

c) Cada word tem o seu único endereço que pode ser acedido individualmente. A unidade de memória mais pequeno que pode ser endereçado é então uma word (sequência ordenada de bytes/bits que não podem ser acedidos individualmente).

4)

a) O barramento de dados é responsável por transferir dados entre o CPU, a memória principal e os módulos de I/O.

b) O barramento de endereços é responsável por transfeir os endereços de memória, onde vai ser escrita ou lida a informação presente no barramento de dados.

c) O barramento de controlo é responsável por especificar o tipo de operação a realizar e assegurar a comunicação entre os vários elementos do sistema computacional, recolhendo também sinais de status.

5) Barramento de endereços

6) Barramento de controlo.

Microcontroladores (Embeded Systems)

7) Um cross-compiler é um programa que corre numa plataforma e gera código para uma plataforma com uma arquitetura diferente daquela onde é executada.

8) A função de um bootloader num sistema baseado em microcontroladores é transferir o código executável a partir de um sistema host, usado no desenvolvimento, para a memória do microcontrolador, permitindo a sua posterior exec.

9)

a) Circuitos integrados com CPU, RAM, ROM, periféricos.

b) Frequências muito inferiores (< 200 MHz) comparado c/ um sistema de uso geral (> 3 GHz)

c) Grande variedade de periféricos

d) Por norma, um custo inferior ao de um sistema de uso geral.

e) Baixo consumo energético, comparado a

f) Utilizado em vários áreas em que a aplicação não necessite de elevados rec. computacionais. / Tarefas específicas.

10) Sistema computacional especializado c/ recursos disponíveis, em geral, mais limitados que num sistema computacional de uso geral. Tem requisitos próprios e executa apenas tarefas pre-def. e é, normalmente, implementado c/ base num microcontrolador.

11) Um microcontrolador PIC32 usa internamente uma arquitetura Harvard com memórias de código e dados independentes e de diferentes tecnologias.

Módulos I/O

12)
a) Configurar o porto, como entrada ou saída.

b) Escrever valores num porto de saída

c) Ler valores de um porto de entrada.

13)
a) Sim

b) Sim

c) RD_LAT

14)
a) Sim

b) RD_TRIS

15)
a) Saída (Registo TRIS está a '0')

b) Está a ser escrito o valor '0' no registo LAT de um determinado porto.

16)
a) Saída (Registo TRIS está a '1')

b) Está a ser feita a leitura do registo PORT (com 2 ciclos de atraso)

17) São utilizados flip-flops tipo D

18) Essa implementação visa resolver os problemas de meta-estabilidade decorrentes do fato de o sinal externo ser assíncrono ao sysclk.

19) Sike, not gonna happen.

Noções Básicas sobre periféricos

20) Módulo I/O

21) O CPU toma a iniciativa de estabelecer o contacto com o periférico e espera que o periférico esteja disponível para a troca de informação (polling). Durante o período de polling o CPU não realiza qualquer operação. Quando o periférico sinaliza o CPU de que está pronto, o CPU inicia e controla a transf. de informação.

22) E/S programada.

23) Referimos-nos à parte que providencia a adaptação entre as características intrínsecas do dispositivo periféricos e as do CPU/memória.

24)

a) Portas tri-state

b) Porque garantem que se lê a informação do periférico (dispositivo E/S) certo, tendo em conta que estas portas só estão ativas quando os sinais CS e RD estão ativos em simultânea, se isto não se verificar a entrada encontra-se em alta impedância.

25)

a) Operação: Leitura
espaço de end.: memória

b) Leitura
I/O

c) Escrita
I/O

d) escrita
I/O

e) escrita
memória

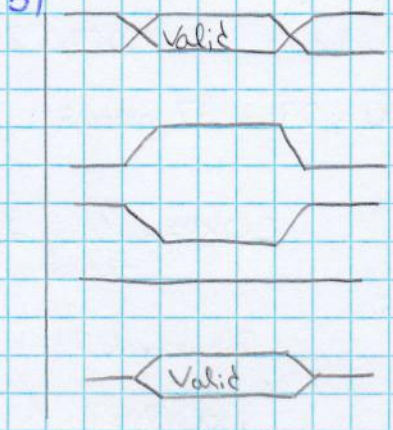
f) leitura
memória

26)

a)



b)



Interrupções

27)

a) O PIC32 tanto pode funcionar em "single-vector mode" (um vetor único p/ todas as fontes de interrupção, ident. p/ software) ou em "multi-vetor mode" (as interrupções são vetorizadas, ident. p/ hardware).

b) Cada periférico possui um identificador único (vetor) e durante o processo de atendimento, na fase da ident. da fonte, o periférico gerador da interrupção identifica-se através do seu vetor. Este vetor vai ser depois usado como índice de uma tabela que especifica como chegar à RSI.

c) O controlador de interrupções permite a configuração da prioridade de cada fonte. Dessa forma a ordem de atendimento depende do valor da prioridade configurada de cada fonte. Este valor está presente nos 3bits do registo IPC, que codificam 7 níveis de prioridade (1 a 7, sendo o '0' fonte desativada).

28)

a) Prólogo → Salvaguarda do contexto do programa que foi interrompido (registos internos do CPU → memória (stack))

b) Epílogo → Reposição do contexto do programa interrompido (memória (stack) → registos internos do CPU)

29) O overhead neste contexto é o custo adicional que o sistema tem para poder realizar transferências de informação por interrupção. Esse overhead é causado pelo número de ciclos gastos na mudança de contexto, prólogo e epílogo.

30) f = 10 MHz
MIPS = 5 ⇒ CPI = 2
overhead = 20 ciclos
RSI = 40 instruções

Overhead + (40 × 2) = 20 + 80 = 100 ciclos

$1 \text{ ciclo} = \frac{1}{10 \times 10^6} = 0{,}1 \times 10^{-6} = 0{,}1\ \mu s$

$100 \times 0{,}1\ \mu s = 10\ \mu s$ $\quad 10\ \mu s = \frac{1}{10 \times 10^{-6}} \Leftrightarrow 0{,}1 \times 10^6\ Hz = 100\ kHz$

31) f = 40 MHz
MIPS = 16 ⇒ CPI = 2,5
fout = 200 kHz
overhead = 75 ciclos

$200\ kHz = \frac{1}{200 \times 10^3} = 0{,}005 \times 10^{-3} = 5\ \mu s$

$5\ \mu s \times 40\ MHz = 200$ ciclos total − 75 = 125 ciclos

$\frac{125}{2{,}5} + \frac{75}{2{,}5} = 50 + 30 = 80$ interrupções.

32) f = 100 MHz
MIPS = 33,3 ⇒ CPI = 3
fout = 500 kHz
overhead = 80 ciclos

$500\ kHz = \frac{1}{500 \times 10^3} = 0{,}002 \times 10^{-3} = 2\ \mu s$

$100\ MHz \times 2\ \mu s = 200$ ciclos − 80 = 120 ciclos

$\frac{120}{3} + \frac{80}{3} = 40 + 27 = 67$ instruções.

33) f = 10 MHz
CPI = 2
RSI = 70 instruções
fout = 50 kHz

$50\ kHz = \frac{1}{50 \times 10^3} = 0{,}02 \times 10^{-3} = 20\ \mu s$

$20\ \mu s \times 10\ MHz = 200$ ciclos − (70 × 2) = 60 ciclos overhead

34) f = 80 MHz
CPI = 2
overhead = 40 ciclos
RSI = 20 instruções

$80\ MHz = \frac{1}{80 \times 10^6} = 0{,}0125 \times 10^{-6} = 12{,}5\ ns/\text{ciclo}$

$40 + (2 \times 20) = 80$ ciclos $\times$ 12,5 ns = 1000 ns = 1 µs

$\frac{1}{1 \times 10^{-6}} = 1\ MHz$

35)
a) Pela ordem que são colocados os periféricos na cadeia, relativamente ao CPU.

b) Interrupt acknowledge ("Inta")

36) 1) Interrupt request
2) Depois de detetar o Ireq ativo o CPU, quando disponível para atender a interrupção, ativa o sinal Inta.
3) O periférico, depois de detetar o sinal Inta ativo, identifica-se colocando no barramento de dados o vetor.
4) O CPU desativa o sinal Ireq e armazena o vetor
5) O CPU a partir do vetor do periférico e da tabela de vetores salta para a RSI.

37) Interrupções vetorizadas – Daisy Chain.

38) Este sistema de interrupções tem apenas 1 entrada de interrupções e 1 única rotina p/ todas as fontes de interrupção. A rotina lê o registo de status de cada uma dos periféricos até encontrar um que tenha gerado pedido de interrupção. Caso hajam vários pedidos em simultâneo, a ordem pela qual os periféricos são "questionados" determina a ordem de prioridade no atendimento.

DMA

39) 1) O DMA ativa o sinal BusReq, pedindo autorização ao CPU p/ ser bus master e espera.
2) Logo que possa, o CPU coloca os seus barramentos em alta impedância e ativa o sinal BusGrant.
3) O DMA ativa o sinal Dack e o DiskCtrl, de seguida, desativa o sinal Dreq.
4) O DMA efetua a transf. (Durante esta fase o CPU está impedido de aceder à memória)
5) O DMA termina a transf. e desativa os sinais Dack e BusReq. Ativa o sinal de interrupção.
6) O CPU após detetar o sinal BusReq desativo, desativa o sinal BusGrant e passa a ser novamente BusMaster.
7) O CPU, logo que possa, atende à interrupção gerada pelo DMA.

40) Retira o pedido para ser Bus Master, libertando os barramentos (desativa os sinais Dack e BusReq) e ativa o sinal da interrupção.

41) O sinal é gerado pelo CPU quando este deteta o sinal BusReq e o CPU está pronto para libertar os barramentos (colocá-los em alta impedância) tornando o gerador do sinal BusReq, Bus Master.

42) Em modo cycle-stealing o CPU só liberta os barramentos nos ciclos em que não acede às memória. O DMA é Bus Master durante 1 ciclo e depois liberta-o.
1) Torna-se Bus Master;
2) Lê 1 palavra do source address;
3) Liberta os barramentos;
4) Espera;
5) Torna-se Bus Master;
6) Escreva uma palavra no dest. add;
7) Liberta os barramentos;
8) Incrementa o endereço e o num. da palavra transf.

43) Enquanto que em modo bloco o DMA assume o controlo dos barramentos até todos os dados terem sido transferidos, em modo "Burst" o DMA transfere rajadas de palavras até atingir um número de palavras pré-programadas ou até o periférico não ter mais informação a transferir. Se os dados não foram todos transferidos, o perif. dá reset no sinal Dreq reiniciando todo o processo de transferência.

44) DMA não dedicado (2 bus cycle p/ palavra); 1 bus cycle = 1 ciclo relógio → não dedicado

a) ctrl 32 bits, f = 500 MHz, 512 words 32 bits
1 ciclo clk: $\frac{1}{500 \times 10^6} = 0{,}002 \times 10^{-6} = 2\text{ns} \times 2 = 4\text{ns}$
512 words × 4 = 2048 ns

b) ctrl 16 bits, f = 1 GHz, 512 words 32 bits
1 ciclo clk: $\frac{1}{1 \times 10^9} = 1\text{ns} \times 2 = 2\text{ns}$ / 16 bits × 2 = 4ns / 32 bits
4ns × 512 = 2048 ns

c) 1024 ns

d) ctrl 16 bits, f = 500 MHz, 2k words 16 bits
1 ciclo clk: $\frac{1}{500\text{MHz}} = 2\text{ns} \times 2 = 4\text{ns}$ / 16 bits
4ns × 2048 = 8192 ns

45) 1 bus cycle = 2 ciclos relógio ; DMA não dedicado

a) ctrl 32 bits
f = 1 GHz
512 words 32 bits

1 ciclo clk $\frac{1}{1\times10^9}$ = 1 ns, 1 bus cycle = 2 ns × 2 = 4 ns

4 ns × 512 = 2048 ns

b) ctrl 16 bits
f = 500 MHz
2k de 32 bits

1 ciclo clk ((2 ns × 2) × 2) = 8 ns × 2 = 16 ns / 32 bits

16 × 2048 = 32768 ns

c) 1 ciclo clk : $\frac{1}{1\times10^9}$ = 1 ns × 2 = 2 ns × 2 = 4 ns × 2 = 8 ns / 32 bits

8 ns × 256 = 2048 ns.

d) 1 ciclo clk : $\frac{1}{500\times10^6}$ = (2 ns × 2) × 2 = 8 ns / 16 bits

8 ns × 2048 = 16384 ns

46) Vai demorar metade do tempo tendo em conta que só precisa de 1 bus cycle por palavra.

47) DMA não dedicado ; cycle-stealing (1 ciclo relógio entre op); 1 bus cycle = 2 ciclos relógio

a) ctrl 32 bits
f = 250 MHz
512 words 32 bits

1 ciclo clk = $\frac{1}{250\times10^6}$ = 0,004 × 10^{-6} = 4 ns

1 palavra = (fetch + wait + deposit + wait) = (4 × 2) + 4 + (4 × 2) + 4 = 24 ns
↳ 1 bus cycle = 2 ciclos clk

512 × 24 = 12288 ns

b) ctrl 16 bits
f = 1 GHz
512 words de 32 bits

1 ciclo clk $\frac{1}{1\,GHz}$ = 1 ns ; 2 ns / palavra 32 bits

1 palavra = fetch + wait + deposit + wait = 2 × 2 + 2 + 2 × 2 + 2 = 12 ns

512 × 12 ns = 6144 ns

c) ctrl 16 bits
f = 500 MHz
2k words de 16 bits

1 ciclo clk $\frac{1}{500\times10^6}$ = 0,002 × 10^{-6} = 2 ns

1 palavra = 2 ns × 2 + 2 ns + 2 ns × 2 + 2 ns = 12 ns

2048 × 12 ns = 24576 ns.

48) DMA dedicado ; modo bloco

a) ctrl 32 bits
f = 500 MHz
512 w de 32 bits

1 ciclo clk : $\frac{1}{500\times10^6}$ = 2 ns → 1 bus cycles

512 bus cycles

b) ctrl 16 bits
f = 1 GHz
4k w de 32 bits

1 ciclo clk 1 ns ⇒ 1 palavra 32 bits = 2 ns
1 bus cycle 2 bus cycles

4k × 2 = 8k bus cycles

c) ctrl 16 bits
f = 1 GHz
512 w 32 bits

1 ciclo clk : 1 ns ⇒ 1 palavra 32 bits ⇒ 2 ns
1 bus-cycle 2 bus cycles

512 × 2 = 1024 bus cycles

d) ctrl 16 bits
f = 500 MHz
1k w 16 bits

1k bus cycles.

49) DMA não dedicado 32 bits — 1 ciclo relógio = $\frac{1}{100\times10^6} = 10\,ns$

$f = 100\,MHz$

Cycle stealing

2 ciclos p/ operação — 1 palavra = 4 bus cycles (fetch + wait + deposit + wait)

wait = 2 ciclos — $= 4 \times (2\times10) = 80\,ns$

1 word = 4 byte $\Rightarrow T_{byte} = \frac{80\,ns}{4} = 20\,ns$; $f_{byte} = \frac{1}{20\times10^{-9}} = 0{,}05\times10^9 = 50\,MB/s$

50)

a) ctrl 32 bits — 1 ciclo relógio = $\frac{1}{120\times10^6} = 8{,}3\,ns$

$f = 120\,MHz$

1 bus cycle = 2 ciclos clk — 1 palavra = $2\times8{,}3 + 4\times8{,}3 + 2\times8{,}3 + 4\times8{,}3 =$

wait = 2 bus cycle = 4 ciclos clk — $= 99{,}6\,ns$

1 word = 4 bytes $\Rightarrow T_{byte} = \frac{99{,}6}{4} = 24{,}9\,ns$; $f_{byte} = \frac{1}{24{,}9\times10^{-9}} = 40\,MB/s$.

b) ctrl 32 bits — 1 ciclo clk = $\frac{1}{80\times10^6} = 12{,}5\,ns$

$f = 80\,MHz$

T_{BC} = 2 ciclos clk — 1 palavra = $2\times12{,}5 + 6\times12{,}5 + 2\times12{,}5 + 6\times12{,}5$

wait = $3\,T_{BC}$ = 6 ciclos — $= 25 + 75 + 25 + 75 = 200\,ns$.

$T_{byte} = \frac{200}{4} = 50\,ns$; $f = \frac{1}{50\times10^{-9}} = 0{,}02\times10^9 = 20\,MB/s$

c) ctrl 16 bits — $T = \frac{1}{120\times10^6} = 8{,}3\,ns$

$f = 120\,MHz$

1 T_{BC} = 2 ciclo clk

wait = $2\,T_{BC}$ = 4 ciclos clk — 1 palavra = $2\times8{,}3 + 4\times8{,}3 + 2\times8{,}3 + 4\times8{,}3$

$= 99{,}6\,ns$ / 16 bits $\Rightarrow 199{,}2$ / 32 bits

$T_{byte} = \frac{199{,}2}{4} = 49{,}8\,ns$; $f_{byte} = \frac{1}{49{,}8\times10^{-9}} \approx 20\,MB/s$

d) ctrl 16 bits — $T = \frac{1}{200\times10^6} = 0{,}005\times10^{-6} = 5\,ns$

$f = 200\,MHz$

1 T_{BC} = 1 cc

wait = 1 T_{BC} = 1 cc — 1 palavra = $4\times5\,ns = 20\,ns$ / 16 bits $\Rightarrow 40\,ns$ / 32 bits

$T_{byte} = \frac{40}{4} = 10\,ns$; $f_{byte} = \frac{1}{10\times10^{-9}} = 100\,MB/s$.

51) DMA não dedicado 32 bits

$f = 100\,MHz$

1 T_{BC} = 2 cc

a) $T = \frac{1}{100\times10^{+6}} = 0{,}01\times10^{-6} = 10\,ns$ — 1 palavra = $2\times10\,ns + 2\times10\,ns = 40\,ns$

$T_{byte} = \frac{40\,ns}{4} = 10\,ns$; $f_{byte} = \frac{1}{10\times10^{-9}} = 0{,}1\times10^9 = 100\,MB/s$.

b) wait = 1 cc

1 palavra = $2\times10\,ns + 10\,ns + 2\times10\,ns + 10\,ns = 60\,ns$ — $T_{byte} = \frac{60\,ns}{4} = 15\,ns$

$f_{byte} = \frac{1}{15\times10^{-9}} = 0{,}067\times10^9 = 67\,MB/s$

c)

a) $T = 10ns$, 1 palavra = $2 \times 10ns = 20ns$

$T_{Byte} = \frac{20ns}{4} = 5ns$ $f_{Byte} = \frac{1}{5 \times 10^{-9}} = 200 MB/s$

b) 1 palavra = $2 \times 10ns + 10ns = 30ns$

$T_{Byte} = \frac{30}{4} = 7,5ns$ $f_{Byte} = \frac{1}{7,5 \times 10^{-9}} = 133,33 MB/s$

Timers

52) $f_{out1} = \frac{f_{in}}{K_1}$

$f_{out} = \frac{f_{out1}}{K_2} = \frac{\frac{f_{in}}{K_1}}{K_2} = \frac{f_{in}}{K_1 * K_2}$

53) Duty-cycle é a percentagem de tempo em que uma determinado sinal está ativo.

Uma aplicação prática da manipulação deste valor é o controlo das rotações de um motor DC.

54) $f_{out} = \frac{f_{in}}{(K+1)}$

a) $f_{out} = \frac{20MHz}{2000} = 20kHz$

b) $f_{out} = \frac{40MHz}{1250} = 32kHz$

c) $f_{out} = \frac{80MHz}{32768} = 2,44kHz$

d) $f_{out} = \frac{2MHz}{1024} = 1,95kHz$

55) A divisão p/ 2 permite obter um duty-cycle de 50 %.

56) $TCLK = 20MHz$

$Prescaler = 4$

$PR = 2499$

$OCk = 834$

$f_{out} = \frac{f_{outpres}}{PR+1} = \frac{\frac{TCLK}{Pres}}{PR+1} = \frac{\frac{20 \times 10^6}{4}}{2500} = \frac{20 \times 10^6}{10000} = \frac{20000000}{10000}$

$= 2kHz$

$T_{out} = \frac{1}{2 \times 10^{-3}} = 50ms$

$OCk = \frac{(PR+1) \times duty-cycle}{100} \Leftrightarrow duty-cycle = \frac{OCk \times 100}{PR+1} = \frac{834 \times 100}{2500} = 33,36\%$

57) $f_{out} = 8Hz$

Timer T2

$PBclk = 50MHz$

a)

$f_{out} = \frac{f_{outpres}}{PR+1} \Leftrightarrow f_{out} = \frac{\frac{PBclk}{Pres}}{PR+1} \Leftrightarrow f_{out} = \frac{PBclk}{Pres \times (PR+1)} \Leftrightarrow Pres = \left\lceil \frac{PBclk}{f_{out} \times (PR+1)} \right\rceil$

$\Leftrightarrow Pres = \left\lceil \frac{50MHz}{8 \times 65537} \right\rceil \Leftrightarrow Pres = 9 \Rightarrow Pres\ min = 16$

b) $f_{out} = \frac{f_{out\,pres}}{PR+1} \Leftrightarrow PR+1 = \frac{\frac{50MHz}{16}}{95} \Leftrightarrow PR+1 = 36764 \Leftrightarrow PR2 = 36763$

58)

a) $k_{pres} = 9 \Rightarrow k_{min} = 64$

b) $85 = \frac{50MHz}{64(PR+1)} \Leftrightarrow PR+1 = \frac{50MHz}{64 \times 85} \Leftrightarrow PR = 9190$

59) $f_{out} = 100Hz$
duty-cycle = 25 %
PBclk = 40MHz

a) $k_{prescaler} \leq \left\lceil \frac{PBclk}{(65535+1) \times f_{out}} \right\rceil = \left\lceil \frac{40 \times 10^6}{65536 \times 100} \right\rceil = 7$

$k_{min} = 8$

b) $PR = \frac{50MHz}{8 \times 100} - 1 \Leftrightarrow PR = 62499$

$OCSRS = \frac{(PR+1)\,dutyCycle}{100} = \frac{62500 \times 25}{100} = 15625$

c) Resolução = $\log_2(62500) = 15,93 = 16$

60) TCLK = 20MHz PR e OCK = ?
k = 8
f_{out} = 200Hz
Duty Cycle = 25 %

$f_{out} = \frac{\frac{TCLK}{k}}{(PR+1)} \Leftrightarrow PR+1 = \frac{TCLK}{f_{out} \times k} \Leftrightarrow PR = 12499$

$OCK = \frac{(12499+1) \times 25}{100} = 3125$

61) PBclk = 20MHz

$f_{out} = \frac{1}{15 \times 10^{-3}} = \frac{1000}{15} = \frac{200}{3}$

$k_{pres} = \left\lceil \frac{50 \times 10^6}{(65535+1) \times 200/3} \right\rceil = 11,42 = 12 \Rightarrow k_{min} = 64$

$\frac{200}{3} = \frac{\frac{50 \times 10^6}{64}}{PR+1} \Leftrightarrow PR+1 = \frac{50 \times 10^6}{64 \times (\frac{200}{3})} \Leftrightarrow PR = 11718,75 = 11719$

62) $f_{out} = \frac{f_{in}}{k+1}$

a) f_{in} = 20MHz
$f_{out} = \frac{1}{5 \times 10^{-3}} = \frac{1000}{5} = 200Hz$

$k+1 = \frac{20 \times 10^6}{200} \Leftrightarrow k+1 = 100000 \Leftrightarrow k = 99999$

b) f_{in} = 25MHz
$f_{out} = \frac{1}{1 \times 10^{-3}} = 1000Hz$

$k+1 = \frac{25000000}{10000} \Leftrightarrow k = 24999$

c) $f_{in} = 40\,MHz$
$f_{out} = \frac{1}{250\times10^{-3}} = 0{,}004 \times 10^{3} = 4\,Hz$

$K+1 = \frac{40000000}{4} \Rightarrow K = 9999999$

63)

a) $f_{in} = 100\,kHz$
counter 16 bits $\Rightarrow$ contar até $2^{16} = 65536$

$T = \frac{1}{100\times10^{-3}} = 10\,\mu s \times 65536 = 0{,}655\,s$

ou

$f_{out} = \frac{f_{pbclk}}{PR+1} = \frac{100\,kHz}{65537} = 1{,}526\,Hz$ $\quad T_{at} = \frac{1}{1{,}526} = 0{,}655\,s$

(Contador incrementa até PR $\Rightarrow$ reset)

b) $f_{in} = 20\,kHz$
counter 10 bits $\Rightarrow 2^{10} = 1024$

$T = \frac{1}{20\times10^{3}} = 0{,}05\times10^{-3} = 50\,\mu s \times 1024 = 0{,}051200\,s$

c) $f_{in} = 50\,MHz$
counter 24 bits $\Rightarrow 2^{24} = 16777216$

$T = \frac{1}{50\times10^{6}} = 0{,}02\times10^{-6} = 20\,ns \times 16777216 = 0{,}3355\,s$

64) $2^{n_{min}} = PR+1 \Rightarrow n_{min} = \log_2(PR+1)$

$f_{in} = 100\,kHz$

$T_{at} = [10\,ms \ldots 170\,ms]$; $f_{out} = [100\,Hz \ldots 5{,}88\,Hz]$

$PR+1 = \frac{f_{in}}{f_{out}} \Rightarrow PR+1 = \frac{100\,kHz}{5{,}88\,Hz} = 17006{,}8027$

$n_{min} = \lceil \log_2(17006{,}80) \rceil = \lceil 14{,}05 \rceil = 15$ bits

65) $f_{in} = 250\,kHz$
$T_{at} = [125\,ms \ldots 480\,ms]$; $f_{out} = [8\,Hz \ldots 2{,}1\,Hz]$

$PR+1 = \frac{250k}{2{,}1} = 119047{,}62$

$n_{min} = \lceil \log_2(119047{,}62) \rceil = \lceil 16{,}86 \rceil = 17$ bits.

66) $T_{at} = [50\,ms \ldots 150\,ms]$; $f_{out} = [20\,Hz \ldots 6{,}67\,Hz]$
$PR+1 = 2^{16}$
$f_{in} = ?$

$PR+1 = \frac{f_{in}}{f_{out}} \Rightarrow f_{in} = (PR+1) \times f_{out} \Rightarrow f_{in} = 65536 \times 6{,}67 = 437125{,}12\,Hz$

Noções Básicas de Barramentos / Descodificação de endereços

67) Address bus = 16 bits
blocos mem = 1 kByte = 10 bits

A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	...	A_0
1	0	0	0	0	0	...	0

a)

i) $CS = A_{15} \cdot \bar{A}_{13} \cdot \bar{A}_{11}$

Lógica positiva: $\bar{A}_{14}^{0} \cdot A_0^{1}$

ii) $CS = \bar{A}_{15} + A_{13} + A_{11}$

" negativa: $A_1 + \bar{A}_0$

b) Gama base: 1000 0000 0000 0000 ... 1000 0011 1111 1111 = [0x8000 ... 0x83FF]

A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	
1	0	0	0	0	0	0x8000 ... 0x83FF
1	0	0	0	0	1	0x8400 ... 0x87FF
1	0	0	1	0	0	0x9000 ... 0x93FF
1	0	0	1	0	1	0x9400 ... 0x97FF

A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	
1	1	0	0	0	0	0xC000 ... 0xC3FF
1	1	0	0	0	1	0xC400 ... 0xC7FF
1	1	0	1	0	0	0xD000 ... 0xD3FF
1	1	0	1	0	1	0xD400 ... 0xD7FF

68) Address bus = 16 bits
blocos mem = 4KByte => 12 bits

$\overline{CS} = A_{15} + \overline{A_{13}} \Leftrightarrow CS = \overline{A_{15}} \cdot A_{13}$

a)

0	0	1	0	0x2000 ... 0x2FFF	(0010 0000 0000 0000 ... 0010 1111 1111 1111)
0	0	1	1	0x3000 ... 0x3FFF	
0	1	1	0	0x6000 ... 0x6FFF	
0	1	1	1	0x7000 ... 0x7FFF	

b) 0x200F ... 0x300F ... 0x600F ... 0x700F

69) Add bus = 16 bits

RAM 1KB = 10 bits => 6 bits seleção : $CS = \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot \overline{A_{12}} \cdot \overline{A_{11}} \cdot \overline{A_{10}}$
Porto : $CS = \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot \overline{A_{12}} \cdot \overline{A_{11}} \cdot A_{10} \cdot \overline{A_9} \cdot A_8 \cdot \overline{A_7} \cdot \overline{A_6} \cdot \overline{A_5} \cdot \overline{A_4} \cdot \overline{A_3} \cdot \overline{A_2} \cdot \overline{A_1} \cdot \overline{A_0}$
dispositivo : $CS = \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot \overline{A_{12}} \cdot \overline{A_{11}} \cdot A_{10} \cdot A_9 \cdot \overline{A_8} \cdot \overline{A_7} \cdot \overline{A_6} \cdot \overline{A_5} \cdot \overline{A_4} \cdot \overline{A_3}$
RAM 2KB = 11 bits => 5 bits seleção : $CS = A_{15} \cdot A_{14} \cdot A_{13} \cdot A_{12} \cdot A_{11}$

70) 0x00102 => 20 bits = k
S = 3 bits e N = 32 bits
N = k + R + S <=> R = 32 - 20 - 3 <=> R = 9 bits.

S: 2^S = sub-gamas / linhas de seleção
R: 2^R = Dimensão de cada sub-gama

	k	S	R
0x00102000 ... 0x00102 1FF	00102	000	1 1111 1111
0x00102200 ... 0x00102 3FF	00102	001	1 1111 1111
0x00102400 ... 0x00102 5FF	00102	010	1 1111 1111
0x00102600 ... 0x00102 7FF	00102	011	1 1111 1111
0x00102800 ... 0x00102 9FF	00102	100	1 1111 1111
0x00102A00 ... 0x00102 BFF	00102	101	1 1111 1111
0x00102C00 ... 0x00102 DFF	00102	110	1 1111 1111
0x00102E00 ... 0x00102 FFF	00102	111	1 1111 1111

71)
a) N = 20 bits
S = $\log_2(4)$ = 2 bits (2^2 = 4 linhas de seleção / sub-gamas)
R = $\log_2(8k)$ = 13 bits (2^{13} = 8k, dim de cada sub-gama)
k = N - S - R = 5 bits

b) Nope

c) 1ª: [0000 0000 0000 0000 0000 ... 0000 0111 1111 1111 1111] <=>
<=> [0x00000 ... 0x07FFF]

2ª: [0x08000 ... 0x0FFFF]

ultima: [0xF8000 ... 0xFFFFF]

d) k S R
1ª memória: [1111 1000 0000 0000 0000 ... 1111 1001 1111 1111 1111] <=>
<=> [0xF8000 ... F9FFF]

2ª mem: [0xFA000 ... 0xFBFFF]

3ª mem: [0xFC000 ... 0xFDFFF]

4ª mem: [0xFE000 ... 0xFFFFF]

e) $0x3AC45 = \underbrace{0011}_{K}\ \underbrace{1010}_{S}\ \underbrace{1100\ 0100\ 0101}_{R}$

endereços da gama: $[0x38000 \ldots 0x3FFFF]$

endereços da memória: $[0x3A000 \ldots 0x3BFFF]$

72)

a)

73)

74) $N = 16$ bits

$\overline{CE} = A_{15} + A_{14} + \overline{A_{12}} \Leftrightarrow CE = \overline{A_{15}} \cdot \overline{A_{14}} \cdot A_{12}$

0 0 0 1 ... $[0x1000 \ldots 0x1FFF]$ $[0x3000 \ldots 0x3FFF]$

75) $N = 32$ bits
$S = \log_2 64 \Leftrightarrow S = 6$ bits
$R = \log_2 16K \Leftrightarrow R = 14$ bits
$K = 32 - 14 - 6 = 12$ bits

a) 12 bits Gama: $0x00000000 \ldots 0x000FFFFF \rightarrow ?$

b) $N = 32$ bits
$S = 6$ bits
$R = \log_2(1K) = 10$ bits
$K = 32 - 10 - 6 = 16$ bits
Gama: $0x00000000 \ldots 0x0000FFFF \rightarrow ?$

76) $f_{in} = 200MHz$
$CPI = 2,5$
10 instrução
32 bits

$T_{in} = \frac{1}{200 \times 10^6} = 0,005 \times 10^{-6} = 5ns/ciclo$

$2,5 \times 10 = 25 \times 5ns = 125ns / 32bits$
$= 4$ bytes

$T_{byte} = \frac{125ns}{4} = 31,25ns/byte$

$f_{byte} = \frac{1}{31,25 \times 10^{-9}} = 32MB/s.$

77) 16 circuitos c/ 1M x 4 bits = 16M x 4 bits

1 byte = 8 bits logo 8M x 8 bits

78) 512 K / 64K = 8 circuitos de 64K x 8 bits

79) N = 16 bits

A15 A14 A13 A12 A11 A10 ...

x x 0 0 0 0

a) $2^2 = 4 \Rightarrow$ 3 réplicas

b) Os espaços de endereçamento são contíguos. Não sei explicar porquê mas acho que é por os 2 bits livres da descodificação estarem à esquerda (MSB) dos 4 bits de selecção e não à direita (LSB).

80)

a) 0x0000 ... 0x07FF $\Rightarrow$ R = 11 bits (0 que selecciona os 9 msb)

0000 0000 0111 1111 1111

0000 0000 0100 0000 0000

$\overline{Sel} = \sum_{i=11}^{19} A_i$

b) $\overline{Sel} = \overline{A_{16}} + \sum_{i=11}^{15} A_i$

c) 16 K $\Rightarrow$ R = K bits 0x8000 ... 0x8FFF

1000 1111 1111 1111 1111

1000 1100 0000 0000 0000

$\overline{Sel} = \overline{A_{19}} + \sum_{16}^{18} A_i + \overline{A_{15}} + \overline{A_{14}}$

d) 0x0000 ... 0x03FF

0000 0011 1111 1111

$Sel = \prod_{i=10}^{15} \overline{A_i}$

e) 0x9000 ... 0x9FFF

1001...

$Sel = A_{15} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot A_{12}$

81)

a) 00x1... d) 0x10

[0x1000 ... 0x1FFF] [0x3000 ... 0x3FFF] [0x20000000 ... 0x2FFFF...]

[0x60000000 ... 0x6FFF...]

b) 0x11...

[0x3000 ... 0x3FFF] [0x7000 ... 0x7FFF]

c) 0x01...

[0x100000 ... 0x 0x 1FFFFF]

[0x500000 ... 0x 5FFFFF]

92) Organizado em barramentos de dados.

a) A prioridade é pré-programada e é atribuída ao elemento de maior prioridade.

b) A prioridade é atribuída de forma a que o último elemento a fazer o pedido seja o primeiro a ser selecionado

c) os elementos são ordenados p/ ordem do pedido e o barramento é atribuído p/ ordem de chegada.

d) Acesso ao barramento é atribuído rotativamente, um master quando termina a transferência passa o controlo ao master seguinte.

93) Protocolo semi-síncrono, opera de forma síncrona mas torna-se "handshaken" quando houver um pedido explícito, por parte do slave, para alterar a temporização das ações.

94) Barramento paralelo de tipo síncrono é caracterizado pela existência de um sinal clock e a transmissão de dados em N linhas.
São suportados transf. do tipo:
- Read-modify-write
- Read-after-write
- Block

95) Transferências assíncronas são caracterizadas pela utilização de sinais protocolo (Valid/Accept) que asseguram a correta troca de informação entre a origem e o destino. Esta flexibilidade temporal permite a comunicação em sistemas c/ diversos dispositivos slaves com diferentes frequências.

96) barramento de 16 bits ; microciclo

a) 2 ciclos/endereçamento + 2 ciclos/transf. de dados

4 ciclos

b) 1 ciclo/endereçamento + 2 ciclos/transf. de dados

3 ciclos

c) 2 ciclos/endereçamento + 1 ciclo/transf de dados

3 ciclos

d) 3 ciclos/endereçamento + 1 ciclo/transf. de dados

4 ciclos

97) Starvation acontece quando um elemento não tem acesso ao barramento devido a constantes pedidos provenientes de elementos de prioridade superior.

98) $f_{in} = 200MHz$
ciclo c/ 10 instruções
CPI = 2,5

$T_{in} = \frac{1}{200 \times 10^6} = 0,005 \times 10^{-6} = 5ns$

10 instrução $\Rightarrow$ 25 ciclos $\Rightarrow$ 125ns/32 bits = 125 ns/4 bytes

$T_{byte} = \frac{125ns}{4} = 31,25ns$; $f_{byte} = \frac{1}{31,25 \times 10^{-9}} = 32MB/s.$

99)

a) Leitura

b) Transferência síncrona

c) Micro-ciclo

90)

a) $f = 500MHz \quad T = \frac{1}{500 \times 10^6} = 0{,}002 \times 10^{-6} = 2ns$ / ciclo

$T_{acc} = 12ns$; $T_{descodificar} = 2{,}5ns$

$T_{total} = 14{,}5ns \approx 15ns / 2ns = 7{,}5$ ciclos

$7{,}5 - 4 = 3{,}5 \times 2 = 7ns$

4 wait-states.

Aula 11 e 12 rever

b) $f = 200MHz \quad T = 5ns$

$T_{validação} = 12{,}5ns$

$T_{acc} = 35ns$; $T_{descodificar} = 7ns$

$T_{total} = 35 + 7 + 12{,}5 = 54{,}5ns / 5ns = 11$ ciclos

Serão necessários 9 wait-states

91)

a) A fase de endereçamento engloba a transf. de dados.

b) O endereçamento e a transf. de dados são tratados como op. autónomas com sinais de controlo e informação separados.

c) Operação atómica, indivisível, que consegue ler uma posição de memória e escrever na mesma posição de memória um valor novo (quer seja completamente novo ou alterado).

d) Permite ler (verificar) após a escrita o mesmo/determinado valor

e) Permite a transf. de um bloco de dados c/ apenas 1 ciclo de endereçamento. A primeira transf. requer um ciclo de endereçamento mas as transf. subsequentes apenas incluem ciclos de dados.

92)

a) esquerda: escrita (dados disponíveis antes do strobe) / direita: leitura (dados disponíveis depois do strobe). Ponto vista CPU

b) Síncrona

c) mux de endereços e dados

d) escrita/leitura c/ protocolo síncrono e endereçamento micro-ciclo em barramento multiplexado.

93) Round-Robin, prioridade - dinâmico (FIFS)

94) a) Ao elemento de maior prioridade.

Barramento de Comunicação Série

95)
a) É mais simples de implementar

b) Menos cablagem

c) Diminuição de custos de interligação

d) Possibilidade de transmissão a longas distâncias

e) Taxas de transmissão mais elevadas

96)
a) barramento inclui sinal do relógio

b) barramento não inclui sinal do relógio mas inclui sinais extra de protocolo para sinalizar princípio e fim de transmissões.

97) Partilhar o mesmo sinal de relógio.
Ex: Relógio explícito do transmissor; Relógio explícito do recetor; Relógio explícito mutuamente - sincronizado; Relógio codificado.

98) A informação transmitida resulta do or exclusivo entre o bit a transmitir com o sinal de relógio.

99) Full-duplex: sistema de comunicação bidirecional em que a transmissão de info. ocorre simultâneamente nos dois sentidos.

Half-duplex: Sistema de comunicação em que a transmissão de informação, ocorre à vez em apenas um sentido.

100)
a) SPI, RS232C, USB 3.x

b) I2C, CAN, USB 1.0 e 2.0

101) Orientado ao bit: inf. organizada em tramas; As tramas são constituídas p/ um símbolo de sincronização seguido de uma sequência de bits.
Ex. CAN; USB

Orientado ao byte: O envio de um byte é uma op. atómica. Podem ser enviados bytes que não serão informação de dados servem para o recetor estruturar inf. recebida
Ex. I2C

Protocolo SPI

102)
a) Master-Slave com ligação ponto-a-ponto.

b) Comunicação síncrona (relógio explícito do master)

c) Comunicação bidirecional full-duplex.

103)
a) $\left(16 \times \frac{1}{100 \times 10^3}\right) \times 20 = 160\,\mu s \times 20 = 3200\,\mu s = 3{,}2\,ms$

b) $\left(8 \times \frac{1}{20 \times 10^3}\right) \times 8 = (8 \times 50\,\mu s) \times 8 = 3{,}2\,ms$

c) $\left(8 \times \frac{1}{100 \times 10^3}\right) \times 10 = (8 \times 0{,}01 \times 10^{-3}) \times 10 = 8 \times 10\,\mu s \times 10 = 80\,\mu s \times 10 = 0{,}08\,ms$

d) $\left(8 \times \frac{1}{50 \times 10^3}\right) \times 30 = (8 \times 0{,}02 \times 10^{-3}) \times 30 = 160\,\mu s \times 30 = 4{,}8\,ms.$

104) Ponto-a-ponto síncrono.

105) Não, o protocolo SPI é adequado para ligações a curta distância apenas (usado principalmente em embedet systems).

106) A saída SDO do master liga-se à entrada SDI do slave0, a saída SDO do slave 0 liga-se à entrada SDI do slave1... a saída SDO do slave2 liga-se à entrada SDI do master. Como o conjunto dos slaves é visto pelo master como um único disp. o conjunto dos 3 slaves têm de ter no máx 8 bits (dimensão do master.)

107) 1) Configurar a frequência do clk
2) Especificar qual o flanco do relógio usado para a transmissão (a receção é feita no flanco oposto).

I2C

108)
a) 0110101
b) leitura
c) 1 Ack (o primeiro)
d) 1 Ack (o segundo)
e) 1 nAck (penultimo bit) gerado pelo master
f) 00000010_10011011 = 0x029B
g) Clock stretch → o slave pode forçar o alongamento da transferência mantendo a zero a linha SCL.

Temos 1 situação de clk stretching gerada pelo slave.

h) $T = \frac{1}{1 \times 10^6} = 1\,\mu s$ $T_{total} = 29\,\mu s.$

109) O endereçamento é feito nos 7 bits (sem contar o start-bit) mais significativos da trama a enviar, seguido do bit que sinaliza o tipo da operação. Cada slave vai ler o endereço e se os endereços corresponderem, o slave vai ler o bit da operação p/ saber se vai receber ou transmitir dados. Por fim, o slave gera um acknowledge e inicia-se a transf. de info.

110) Um barramento I2C é composto p/ 2 linhas físicas, SDA (serial data out) e SCL (serial clock line)

111) Um barramento está livre quando não estão a ser feitas transf., ou seja, após um STOP bit.

112) O master controla a linha SCL.

113) Na gestão de acesso ao barramento é necessário primeiro garantir que os clocks dos masters estão sincronizados recorrendo à técnica bit dominante/recessivo. Depois, por cada novo bit enviado, quando a linha SCL está a '1' cada master lê a linha SDA e verifica se o seu valor coincide com o que enviou. O processo de arbitragem é perdido p/ um master quando lê da linha "0" tendo enviado "1".

114) Nível lógico '0' chama-se bit dominante
Nível lógico '1' chama-se bit recessivo

Na presença do nível lógico '0' o barramento é forçado a '0' independentemente do o que lá estava, sendo p/ isso chamado bit dominante.

115) O slave pode forçar o alongamento da transf. mantendo a zero a linha SCL. O sinal "wait" condiciona o estado da linha SCL do barramento, enquanto wait='1' ⇒ SCL='0' assim slaves mais lentos garantem que a transf. é sempre concluída c/ sucesso.

RS232

116) Ligação ponto-a-ponto. Na sua forma mais simples requer a utilização de 2 linhas de sinalização e uma de massa

b) Comunicação bidirecional, full-duplex.

c) Comunicação assíncrona c/ relógio implícito.

d) Bit oriented

117) Erro de paridade: Quando o bit de paridade, por exemplo, devia ser '0' e é lido como '1'.
Erro de framing: Quando é detetado o nível lógico '0' no instante em que era esperado um stop bit (nível lógico '1')

118) Para diminuir o erro de fase é utilizado uma técnica que consiste em gerar no recetor um relógio com uma freq. N vezes superior ao relógio do transmissor e sincronizar a receção a partir desse relógio. Por ex. se N=16 o erro de fase max. vai ser T/16, em que Te é o período do relógio do transmissor. O erro de fase mantém-se até ao final da receção de trama corrente, mas os instantes de validação estão bem definidos (no mesmo ex. "Start bit" valido ao fim de 8 ciclos de relógios e os restantes bits validados a cada 16 ciclos de relógio).

119) Baudrate 57600
trama é composto p/ 8 bits c/ paridade par
2 stop bits
fator de sobre amostragem de 16.

a)

120) N=6k ; 38400bps ; 7 bits sem paridade ; 1 stop bit

a)

121)

a) 19200 bps
7 bits de dados
1 bit de paridade
1 stop bit
(1 start bit)
10 bits total

$\frac{19200 \times 7}{10} = 13440$ bps.

b) $\frac{115200 \times 8}{11} = 83781,82 \approx 83782$ bps

c) $\frac{9600 \times 8}{12} = 6400$ bps

d) $\frac{1200 \times 7}{9} = 933,33 = 933$ bps

122) 100000 bps
8 data bits
paridade ímpar
1 stop bit

0x5A
0101 1010



123) 7 bits de dados
paridade par
2 stop bits
Tx

8 data bits
sem paridade
1 stop bit
Rx

Não vai ser detetado nenhum erro no recetor.

0x2D,

124) 8 data bits
ímpar
1 stop bit
Tx

7 data bits
s/ paridade
1 stop bit
Rx

Vai ser detetado um erro de framing, é detetado um '0' onde deveria ser detetado um stop bit ('1').

125) 7 data bit
par
2 stop bits
Tx

Device Drivers

126)

a) O device driver é um programa que permite a outro programa/app/SO interagir com um dado dispositivo hardware. Para isso implementa uma camada de abstração que lida com as particularidades do dispositivo controlado.

b) A função do device driver é traduzir as chamadas da aplicação ou SO, em ações específicas do dispositivo. Posto isto, cria-se uma independência entre o hardware e a aplicação/SO.

127) O device driver tem de lidar com aspetos específicos da implementação do dispositivo por isso o seu fornecimento é assegurado pelo fabricante. A biblioteca de classes para comunicação tem de ser compatível para que não haja necessidade de alterar a aplicação.

128)

a) Um buffer circular é um array dinâmico c/ variáveis de acesso definidas, que são incrementadas ou decrementadas em cada iteração. Um FIFO pode ser implementado c/ um buffer circular.

b) Head, tail e count. A primeira aponta p/ o caracter que está no buffer à mais tempo, pronto a ser transmitido. A segunda aponta p/ a primeira posição livre do buffer, onde vai ser escrita a informação nova. A variável count expressa o número de caracteres presente no buffer.

c) A variável count é considerada um recurso partilhado pois ao contrário das variáveis head/tail que têm de atualizadas na rotina ou na função, dependendo do sentido da transferência, a ~~not~~ variável count tem de ser atualizada a cada iteração para que quer a rotina quer a função tenham a info. do estado do buffer atualizado. Para que o incremento desta variável ocorra dentro da normalidade, esta operação tem de estar numa secção crítica de código.

129) As secções críticas servem p/ garantir que o incremento da variável count (que ocorre na função) é uma operação atómica. Desta forma procede-se à desativação das interrupções, para que quando ocorrer este incremento a operação não seja interrompida para executar outra zona de código que também possa alterar o valor dessa variável. Após ocorrer o incremento, ativa-se de novo as interrupções e finda a secção crítica.

CAN

130)

a) Barramento multi-master. Número de ligações físicas limitadas pelo número possível de identificadores diferentes.

b) Bidirecional half-duplex

c) Assíncrono com relógio implícito

d) Bit oriented.

131) A técnica de bit-stuffing é utilizada p/ garantir um tempo máximo entre transições da linha de dados, assegurando que os relógios locais estão sincronizados.
Consiste em introduzir um bit de polaridade oposta, a cada 5 bits iguais. Este bit é descartado pelo recetor.

132) "Identifier" é responsável por identificar a mensagem transmitida. Este campo é também utilizado p/ fazer a arbitragem do acesso ao barramento, sendo o id mais baixo o de maior prioridade.

133) Qualquer nó pode desempenhar o papel de master mas não podem ser enviadas 2 mensagem em simultânea, para evitar estas situações há o processo de arbitragem q garante que o barramento é entregue a um master de cada vez.

134) A arbitragem é feita analisando o campo identifier bit-a-bit, o identifier de menor valor ganha o processo.

135) Todos, porque a comunicação é do tipo broadcast.

136) CRC error: CRC calculado ≠ CRC recebido.

Acknowledge error: o produtor não recebe um bit dominante no campo ACK, o que significa que a mensagem não foi recebida p/ nenhum nó da rede.

Form error: verificação que analisa campos que devem de estar sempre a '1'.

Bit error: Cada bit transmitido é analisado pelo produtor da mensagem: se o produtor lê um valor que é o oposto do que escreveu gera um erro.

Stuffing error: Se após 5 bits iguais não surgir um de polaridade oposta, é gerado um erro.

137) máscara = 0111 1111 1010
filtro = 0101 1100 0000

138)

139) Data frame: usado no envio de dados de um produtor para os recetores.

RTR Frame: Enviado a '1' por um nó consumidor a solicitar uma transmissão de uma trama de dados específica.

Error frame: Usado para reportar um erro detetado.

Overload frame: usado para atrasar o envio da prox. trama em caso de sobrecarga.

USB

40)
a) Não periódica, sem garantia de taxa de transmissão ou latência (largura de banda). Garante entrega (??). Info. a transmitir de pequenas dimensões.
Utilizado pelo host-controller p/ envio de mensagens e comando e configuração ou consulta de estado

b) Não periódica, sem garantia de taxa de transmissão, latência ou largura de banda.
Garante a entrega c/ mecanismo de retransmissão em caso de deteção de erros.
Utilizado em discos/pen-drives...

c) Periódicas com limite máximo p/ latência, largura de banda garantida.
Erros não são corrigidos, não há controlo.
Utilizado em aplicações de audio/video.

d) Periódica c/ limite max. p/ latência. Retransmissão em caso de erro e é utilizado p/ transmitir baixas quantidades de informação. Ratos, teclados...

41) Ligação física em árvore com um máximo de 7 níveis e de 127 dispositivos, partindo de um sistem central p/ vários dispositivos.

42)
a) Master-slave

b) USB 1.0 e 2.0 é half-duplex
e 3.0 full-duplex

c) Codificação: NRZI
Os dados são transmitidos em modo diferencial em par entrançado.

d) Comunicação síncrona orientada ao bit.

43) Tanto pode ser um hub e disponibilizar acessos para outros dispositivos, ou então, pode ser um extremo, onde o dispositivo encarado como um simples periférico.

44) Tarefa permanente que permite ao host-controller reconhecer e inicializar um dispositivo (Atribuição de um endereço, leitura de info., alocação de recursos, atribuição e carregamento de Device-Driver)

45) Endereço do dispositivo, identificador do dispositivo e pode ir de 1 a 127.

46) Número do endpoint, corresponde ao terminal lógico associado e pode ir de 0 a 15.

Direção, in ou out.

47) Pipes, estabelecem a comunicação entre o host controller e um endpoint. Permite comunicação bidirecional de forma a conectar o host c/ o dispositivo.

Tecnologia, Organização e Funcionamento de RAM

47) SRAM:
Pelo menos uma ordem de grandeza mais rápido
Não necessita de refrescamento constante
Custo por bit elevado
Maior dissipação de potência

DRAM:
Mais lenta
Necessita de refrescamento
Custo p/ bit menor
→ Menor dissipação de potência

148) A organização em matriz aumenta o número de decodificadores mas reduz o número total de portas lógicas. Cada palavra pode ser vista como um elemento da matriz e n como uma linha ou memória como na organização linear.

149) Tempo total da operação de leitura

150)

a) Tempo necessário para os dados ficarem disponíveis no barramento de dados

b) Taxa a que os dados podem ser transferidos de/para memória.

151)

a) $\frac{128k}{32k} = 4 \times \frac{8}{1} = 32$ circuitos

b) $\frac{128k}{32k} = 4 \times \frac{8}{4} = 8$ circuitos

c) $\frac{128k}{16k} = 8 \times \frac{8}{8} = 8$ circuitos

d) $\frac{128k}{64k} = 2 \times \frac{8}{8} = 2$ circuitos

e) $\frac{128k}{128k} = 1 \times \frac{8}{1} = 8$ circuitos

152) $256k = 2^{18}$

2^9 linhas $\times$ 2^9 colunas = 256k células

512 linhas, 512 colunas, 8 matrizes

153)

a) $16M \times 4$

b) $8M \times 8$

c) $2M \times 32$

d) $2M \times 32$

e) $8M \times 8$

154) $512M = 2^{29}$ (Total) $29 - 14 = 15$ bits p/ colunas
$16k = 2^{14}$ (linhas)

$\max(14, 15) = 15$ bits,

b) $256M = 2^{28}$ (Total) $28 - 14 = 14$ bits (colunas)
$16k = 2^{14}$ (linhas)

c) $4G = 2^{32} \Rightarrow 16$ bits p/ colunas e linhas

d) $1G = 2^{30}$ (Total)
$32k = 2^{15}$ (linhas) $\Rightarrow$ 15 bits p/ colunas

e) $2G = 2^{31}$ (Total)
$64k = 2^{16}$ (linhas) $\Rightarrow$ 15 bits p/ colunas

barramento = $\max(15, 16) = 16$ bits

f) $256M = 2^{28}$
8K linhas = 2^{13} => 15 bits p/ colunas Barramento = max(13,15) = 15 bits

155) $2M = 2^{21}$ (total)
$2K = 2^{11}$ (colunas
linhas = $2^{10} = 1K$

a) $1024 \times (50+25) = 76,8$ ms

b) $1024 \times (40+15) = 1K \times 55$ ns $= 56,32$ ms

c) $1024 \times (65+30) = 1K \times 95 = 97,28$ ms

156)
a) $8M = 2^{23}$; $1024 = 2^{10}$ => linhas = $2^{13} = 8K$

$8192 \times (50+25) = 8K \times 75$ ns

b) 16K linhas

$16K \times (50+25)$

c) $4M = 2^{22}$; $4096 = 2^{12}$ => linhas $2^{10} = 1024$

$1K \times (50+25)$

d) $8M = 2^{23}$; $1024 = 2^{10}$ => 2^{13} linhas = 8K

$8K \times (50+25)$

157) 1- Pré-carregar a linha "bit" a VDD/2
2- Ativar a linha select
3- Valor lógico detetado pela diferença de tensão na linha bit, relativamente a VDD/2
4- Restauro do valor do condensador.

158) (Verificar sinal "WE")

Leitura em page mode

Memória cache.

159)
a) (mapeamento direto) 1 comparador

b) " 1 comparador

c) parcialmente associativa (assoc. 4) 4 comparadores

d) " " (assoc. 8) 4 comparadores

e) (totalmente associativa) 1 comparador p/ linha

nº linhas = $\frac{\log_2(256K)}{\log_2(256)} = \frac{2^{18}}{2^8} = 2^{10} = 1024$ comparadores

160) bus add = 32 bits
a) blocos 64 bytes => byte = $\log_2(64) = 6$ bits Tag = 32 - 8 - 6 = 18 bits
256 linhas => group = $\log_2(256) = 8$ bits

b) blocos 128 bytes => byte = 7 bits Tag = 32 - 8 - 7 = 17 bits
256 linhas => group = 8 bits

c) blocos 64 bytes => byte = 6 bits Tag = 32 - 8 - 6 = 18 bits.
$\frac{\log_2(64K)}{\log_2(64)} = 2^{10}/4 = 256$ linhas => group = 8 bits

d) blocos 64 bytes => byte = 6 bits Tag = 18 bits
$\frac{\log_2(128K)}{\log_2(64)} = \frac{2^{17}}{2^6} = 2^{11}/8 = 256$ linhas => group = 8 bits

e) blocos de 256 bytes => byte = 8 bits Tag = 32 - 8 = 24 bits.

161) Há cache totalmente associativa, qualquer bloco de mem. principal pode residir em qualquer posição da cache, enquanto que numa cache de mapeamento direto apenas um bloco de um dado grupo pode residir na cache.
Por outro lado a cache de mapeamento direto de muitos menos comparadores, sendo a implementação mais simples e mais barata.

162) FIFO → é subs. o bloco que está na cache há mais tempo
LRU → é subs o bloco que está há mais tempo sem ser referenciado
LFU → é subs. o bloco menos acedido.
Random → subst aleatória.

163) Write-Through: Todas as escritas são realizadas simult. na cache e na memória principal.
Write-Back: Valor escrito apenas na cache, só é feita a escrita na memória quando o bloco da cache foi substituído.

164) O dirty bit sinaliza se um bloco de mem foi ou não modificado. Assim quando um bloco de memória é subst. o dirty bit é verificado para determinar se é necessário ou não escrever o bloco na mem. principal antes de ser subs. ou se pode ser removido.

165)
a) linhas = $\frac{\log_2(512K)}{\log_2(32)} = \frac{2^{19}}{2^5} = 2^{14}/4$ linhas = $256 = 2^8$; group = 8 bits.

b) linhas = $\frac{2^{16}}{2^6} = 2^{10}/8 = 128 = 2^7$; group = 7 bits

c) linhas = $\frac{2^{11}}{2^1} = 2^{13}/4 = 2048 = 2^{11}$; group = 11 bits.

d) linhas = $\frac{2^{14}}{2^3} = 2^{13}/4 = 2048 = 2^{11}$; group = 11 bits

166) O valid bit indica se a linha correspondente tem conteúdo válido ou se não está ocupada. Para uma cache c/ N linhas é necessários N valid bits.

167) O princípio de localidade consiste no acesso à memória de forma não aleatória, ou seja, os programas os programas levam usar endereços que se situam vizinhança.
Localidade temporal: Endereços acedidos, têm mais probabilidade de serem novamente acedidos num futuro próximo
Localidade espacial: Endereços contíguos ao acedido também têm mais probabilidade de serem acedidos.

168) a) 0,9 x 6ns + 0,1 x 66ns = 12ns
b) 0,95 x 4ns + 0,05 x 39ns = 5,75ns
c) 0,85 x 5ns + 0,15 x 55ns = 12,5 ns.

169)

a) $0,9 \times 2 + 0,1 \times 102 = 12$ ciclos

b) $0,95 \times 2 + 0,05 \times 82 = 6$ ciclos

c) $0,85 \times 4 + 0,15 \times 124 = 22$ ciclos

170) Parcialmente associativa 8KB
associat. 4
blocos 32 bytes $\Rightarrow$ 5 bits (byte)

a) $0x12B8 = 0001\ 0010\ 1011\ 1000$
byte.

$\frac{2^{13}}{2^5} = \frac{8}{4} = 2^6 \Rightarrow$ 6 bits (set) $010101_2 = 15_{16} = 1 \times 16^1 + 5 \times 16^0 = 21_4$

b) $0x355F = 0011\ 0101\ 0101\ 1111$

$\hookrightarrow 101010_2 = 2A_{16} = 2 \times 16^1 + A \times 16^0 = 42_4$

c) $0x2760 = 0010\ 0111\ 0110\ 0000$

$\hookrightarrow 111011_2 = 3B_{16} = 3 \times 16^1 + 11 \times 16^0 = 48 + 11 = 59_{10}$

171) blocos 32 bytes $\Rightarrow$ 5 bits (byte)

$\frac{2^{14}}{2^5} = \frac{2^9}{2^3} = 2^6 \Rightarrow$ 6 bits (set)

a) $0x56B9 = 0101\ 0110\ 1011\ 1001_2$

$\hookrightarrow 110101_2 = 35_{16} = 3 \times 16^1 + 5 \times 16^0 = 48 + 5 = 53_{10}$

b) $0x7041 = 0111\ 0000\ 0100\ 0001_2$

$\hookrightarrow 02_{16} = 0 \times 16^1 + 2 \times 16^0 = 2_{10}$

c) $0x23F2 = 0010\ 0011\ 1111\ 0010_2$

$\hookrightarrow 1F_{16} = 1 \times 16^1 + 15 \times 16^0 = 31_{10}$

172) Campo Tag

173) $\log_2$ (linhas).

174) Add Bus = 16 bits

a) bloco 16 byte $\Rightarrow$ 4 bits (byte)

$$\text{linhas} = \frac{\frac{2^{16}}{2^4}}{4} = \frac{2^{12}}{2^2} = 2^{10} = 1024 \text{ linhas} \Rightarrow \text{group} = 10 \text{ bits}$$

$16 - 10 - 4 = 2$ bits (Tag).

b) byte = 6 bits
group: $\frac{2^{17}}{2^6} = \frac{2^{11}}{8} = 256 \Rightarrow$ 8 bits

Tag = ~~32~~ 16 − 8 − 6 = 2 bits

c) byte = 5 bits
group: $\frac{2^{18}}{2^5} = \frac{2^{13}}{2^2} = 2^{11}$ = 11 bits

Tag = 16 − 11 − 5 = 0

d) byte = ~~8~~ 5 bits
group = $\frac{2^{20}}{2^5} = \frac{2^{15}}{2^4} = 2^{11}$ = 11 bits

Tag = 16 − 11 − 5 = 0 bits

175) Add bus = 16 bits

a) n: 16 colunas $\Rightarrow$ 4 bits | dimensão: $2^{k\,(colunas)} \times 2^{k\,(linhas)} = 1k$

k: 16 − 4 = 12 bits

b) 0x3785 = 0011 0111 1000, 0101 $\Rightarrow$ hit (19)
↳ 5

0xF0A3 $\Rightarrow$ hit (C1) 0x0FB7 $\Rightarrow$ miss

0x1932 $\Rightarrow$ miss 0x59E5 $\Rightarrow$ miss

0x6D51 $\Rightarrow$ hit (9E) 0x0ACF $\Rightarrow$ hit (5D)

Memória Virtual

176) Endereçamento Virtual

a)
dimensão pag virtual: 8kB = $2^{13} \Rightarrow$ VPO = 13 bits
entradas tb c/ processo: 256k = $2^{18} \Rightarrow$ VPN = 18 bits

add bus Virtual = 13 + 18 = 31 bits.

b) 4kB = $2^{12} \Rightarrow$ VPO = 12 bits
1M = $2^{20} \Rightarrow$ VPN = 20 bits

add bus virtual = 32 bits

c) 4kB = $2^{12} \Rightarrow$ VPO = 12 bits
256k = $2^{18} \Rightarrow$ VPN = 18 bits
30 bits

d) 16kB = $2^{14} \Rightarrow$ VPO = 14 bits
128k = $2^{17} \Rightarrow$ VPN = 17 bits
31 bits

177) A TLB é uma memória rápida, semelhante a uma cache, onde se encontram as entradas da tabela mais recentemente utilizada.
Implementada como uma memória associativa com procura paralela.

178)

a) impossível (ñ pode dar miss na page table)

b) possível (A página está na memória, dados ñ disponíveis na cache).

c) impossível

d) impossível

e) possível (pag. disponível na TLB

179) 2^{VPN}

180) Processo no qual o endereço virtual gerado pelo CPU é traduzido num endereço físico da mem. principal. (VPN → PPN + offset)

181)

a) entradas: $512K = 2^{19}$ → offset

dimensão da pag. = 32 - 19 = 13 bits

b) $2M = 2^{20}$

dim pag. = 48 - 20 = 28 bits

c) $128K = 2^{18}$

dim da pag = 30 - 18 = 12 bits

182) 2^{VPN}

183) O valid bit de uma entrada da page table sinaliza que a página pretendida está na memória e que a page table fornece o número da página física.

184) A memória é associativa c/ procura paralela, ou seja, a TLB vai simultâneamente, comparar o VPN com todas as tags das entradas.

185/186) Permite a utilização eficiente da mem. física, nomeadamente.

187) Quando ocorre um page fault

188) LRU → F subs. a página que está lá mais tempo sem ser referenciada
As páginas candidatas têm o reference bit a 0, e 5 copiadas para o disco antes da subs.

189) Write-back → consiste na escrita em disco página a página, e só quando a página necessita de ser retirada da mem. física. O dirty bit indica quando é que a página foi alterada e necessita de ser escrita em disco aquando da sua subst.

190) endereçamento virtual: 26 bits
pag. 512 bytes
Page table → endereços. multiplos de 2
→ 16 bits de dimensão.

a) VPO $\log_2(512) = 9$ bits
VPN = 26 - 9 = 17 bits

191)

188) LRU → F subs. a página que está há mais tempo sem ser referenciada
As páginas candidatas têm o reference bit a 0, e são copiadas para o disco antes da subs.

189) Write-back → consiste na escrita em disco página a página, e só quando a página necessita de ser retirada da mem. física. O dirty bit indica quando é que a página foi alterada e necessita de ser escrita em disco aquando da sua subst.

190) endereçamento virtual: 26 bits
pag. 512 bytes
Page table → endereços multiplos de 2
→ 16 bits de dimensão.

a) VPO $\log_2(512) = 9$ bits
VPN = 26 - 9 = 17 bits

191)